MAKERERE UNIVERSITY
COLLEGE OF COMPUTING AND INFORMATION SCIENCES
SCHOOL OF COMPUTING AND INFORMATICS TECHNOLOGY
DEPARTMENT OF NETWORKS

BSE4104 Emerging Trends in Software Engineering
AI Agentic Systems Capstone

University Student-Support Case Agent
Final Engineering Report

Group M Evening
Namata Sumayya (23/U/22615)
Nakaye Hildah (23/U/13598/PS)
Mushabe Moses (23/U/12131/EVE)
Aryampa Joshua (23/U/06517/PS)

Course convener: Dr Kamulegeya Grace B
23 October 2026

Table of Contents

1. Executive Summary....1
2. Problem Context and Requirements....1
3. System Architecture....2
4. Foundation Model and Prompt Engineering....2
5. Context Engineering and RAG....3
6. Tools and Agentic Workflow....3
7. Memory, State and Interoperability....4
8. Evaluation, Observability and Guardrails....5
9. Software Engineering Quality and Deployment....6
10. Results, Limitations and Professional Reflection....7
References....8
Appendix A. Evidence index....9
Appendix B. Week 8 retest....9

BSE4104 Group M Evening | Final Engineering Report

1. Executive Summary

New Makerere University students need a reliable way to complete the first onboarding steps: webmail, campus Wi-Fi, the ACMIS portal, and registration. The public instructions exist, but they are spread across several documents. A general chat model can sound confident while inventing a step that is not in those documents. The group built a bounded support agent for freshman onboarding only. It answers from an approved corpus, cites the source, and, when the student agrees, saves one support draft for a staff queue. It does not decide admissions, grades, discipline, or fees.

The system uses gemini-3.6-flash for answers and gemini-embedding-001 for retrieval. Two tools are available: retrieve_documents and create_support_case. The application, not the model, executes tools and enforces a three-round limit, a three-tool-call limit, and one draft per session. Week 3 retrieval passed 11 of 15 questions (73%). Week 7 passed 29 of 36 scenarios (80.5%). Week 8 fixed five defects from that evaluation. The principal remaining limit is retrieval: some relevant fee and joining passages still score below the 0.70 similarity cutoff, so the agent refuses rather than guess.

2. Problem Context and Requirements

The user is a freshman who is trying to activate an account or follow a published onboarding step. The workflow is short. The student asks a question. The agent searches the approved documents. If a passage is relevant, the answer is grounded in it and the source title is shown. If nothing relevant is found, the agent says so and may offer a support draft. The student must click "I agree to save a support draft" before a draft can be written. Staff still own the case. The agent cannot resolve, close, or reassign it.

The requirements are recorded in the Agent Task Contract, the user stories and acceptance criteria, and the prompt specification. Success means a grounded answer with a source, a correct refusal when the question is outside freshman onboarding, or a validated draft in the right queue. A wrong success is an invented procedure, a fee or grade decision, a draft saved without agreement, or a second draft in the same session.

The AI boundary is deliberate. The model may draft language and choose a tool. It may not change a university record, approve a refund, alter a grade, or speak for admissions or a disciplinary body. High-impact actions stay with a person. The corpus is public university guidance plus the group's controlled copies. The agent does not connect to ACMIS, the fees ledger, or any live student database.

Acceptance is checked against behaviour, not against a pleasant tone. A webmail question passes when the reply follows a retrieved passage and names the source. A continuing student passes when the reply refuses and does not search. A draft passes when the sidebar agreement is on, the summary is a real sentence, the registration number matches the

freshman pattern, and the stored row has a queue. It fails when any of those is missing, when a second draft is written in the same session, or when the summary asks for a waiver or a refund. These checks are the acceptance criteria the Week 7 scenarios and the deterministic script both use.

3. System Architecture

The running application is a Streamlit page, src/app.py. The student picks a prompt version in the sidebar. Versions v1.0 and v1.1 call the model with no documents. Version v1.2 adds retrieval. Version v1.3 is the agent. For v1.3 the page calls run_tool_turn in src/tools/orchestrate.py. Session data lives in SessionState: session id, conversation, agreement flag, active case id, retrieved context, iteration count, and the tool log.

Deterministic code surrounds the model. Before any model call, Sense checks the current student message for an instruction-override attempt (stop S6), a statement that the student is not a freshman (stop S4), and a blocked decision topic (stop S5). Only a message that passes those checks is sent to Gemini, together with the tool declarations. If the model requests a tool, handlers.py runs retrieve_documents or create_support_case. The model never writes the draft file itself.

Retrieval is local. Chunks and embeddings are files under data/chunks/. Drafts are a JSON file under data/cases/, which is not committed. Each v1.3 turn also writes one JSON log under logs/, which is not committed. Sample logs are kept in evidence/traces/. The external dependency is the Gemini API, reached with a key in .env. There is no other network tool, browser, or payment integration.

The boundary between components is the reason the system can be tested without a live model. Sense, validation, and the draft store are ordinary Python. The deterministic suite substitutes a fake Gemini client and a temporary draft file. That is how C07, C08, R04, and the limit checks can fail or pass on a machine with no API key. The model boundary is also where a 503 stops. The orchestration layer catches it, writes the raw text into the turn log, and returns a short sentence to the page. The student never has to interpret an HTTP status.

4. Foundation Model and Prompt Engineering

The group selected Gemini because the course work needed both a chat model and an embedding model from one provider, with a free-tier key that every member could use. The answer model in the release is gemini-3.6-flash. Embeddings use gemini-embedding-001. The choice is an engineering trade-off, not a claim that this model is the best general assistant. It is small enough to call from a student project, and the application treats its output as untrusted until the tools and checks have run.

Prompts are files, not strings hidden in the interface. prompts/v1.0_baseline.txt is an unconstrained baseline. v1.1 limits the role to freshman onboarding. v1.2 requires answers

from retrieved passages and a refusal when retrieval is empty. v1.3 adds the two tools, the agreement rule, and the draft rules. load_prompt reads the version the sidebar selects. The Week 7 and Week 8 release run v1.3.

The important iteration was to stop asking the model to be the safety system. Early prompts said the assistant must not decide fees or grades. That sentence is still in the prompt, but the release also blocks those topics in code before the model is called, and it rejects fee-decision language inside create_support_case no matter which category the model picked. Prompt text is guidance. The stop conditions are the control.

5. Context Engineering and RAG

The corpus is eight public Makerere freshman-onboarding documents, listed in the corpus source register. The pipeline is download_sources.py, extract_text.py, chunk_text.py, and embed_chunks.py. Chunks are about 1,500 characters. The Week 3 index held 90 chunks. chunks.json is committed so the text can be inspected. The embedding file is not committed, because it is large and can be rebuilt. The live timetable site is excluded. It is not a stable document.

retrieve.py ranks chunks by cosine similarity and returns at most three chunks at or above 0.70. The threshold was chosen after comparing 0.50, 0.65, and 0.70 on the Week 3 questions. A lower bar returned more passages and also more weak matches. The agent would rather say that it did not find a source than quote a weak one.

Week 3 scored 11 of 15 (73%). Answerable questions passed 4 of 5. Unanswerable questions passed 5 of 5, which means the refusal path worked when the corpus did not contain the answer. The misses were mostly threshold misses: joining instructions, Eduroam troubleshooting, webmail password reset, and a combined Wi-Fi question. Week 7 scenario I02 is the same class of miss. A question about whether functional fees are refundable did not clear 0.70, so the agent refused. Week 8 did not lower the threshold and did not add a keyword index. Either change would need the 15-question set, and the full scenario set, to be scored again.

Grounding is a rule in the reply, not only in the retriever. When status is ok, the answer is supposed to follow the passages and name the source. When status is no_match, the answer must not fill the gap from general knowledge about universities. The stop for that path is S3. The student can still be offered a draft, and that draft records the question. It does not record an answer the documents did not support. This is why a retrieval miss is classified as a remaining limit rather than as a silent success. The student is told the document was not found. A staff member can still answer the fee question from the policy itself.

6. Tools and Agentic Workflow

retrieve_documents takes a query and returns status, a count, and source titles with

similarity. It does not return the whole corpus to the student. create_support_case takes a category, an issue summary, and the student registration number when one was given. It validates the registration number against the freshman pattern, rejects an empty or placeholder summary, and rejects any summary that asks to waive, refund, or approve a fee. Valid drafts are stored locally with an id of the form CASE-year-six digits and a queue: webmail and Wi-Fi to the ICT helpdesk, registration to the registration office, fee information to fees information, accommodation to the accommodation office, and anything else to general onboarding.

The loop is Sense, Plan, Act, Observe, Stop. Sense is the code checks above. Plan is the model, which may request a tool. Act is the handler. Observe feeds the tool result back for another round. Stop ends the turn with one of eight labels. S1 is a grounded answer. S2 is a saved draft and the hand-off to staff. S3 is no matching document, with an offer to draft a case. S4 is out of freshman scope. S5 is a blocked topic. S6 is an instruction-override attempt. S7 is the three-round limit. S8 is a tool error after one retry.

Limits are constants in orchestrate.py: three rounds, three tool calls per turn, two case calls per turn so one validation error can be corrected, and one successful draft per session. A second create for the same registration number is refused by the case history check. The agreement control is the sidebar button. Typing "I agree" in the chat is not enough, because the flag is set only by that button. Week 5 traces T1, T2, and T3 show a grounded answer, a draft, and a failure recovery under these rules.

A registration number is accepted only when it matches the freshman form used in the case store: two digits, a slash, U or X, a slash, four or five digits, and an optional /PS or /EVE suffix. The number is read from the student's own messages when the draft is tied to a person. The model is not allowed to invent one. Queues are a fixed map, not a free-text field. Webmail and Wi-Fi go to the ICT helpdesk. Registration goes to the registration office. A fees information question goes to fees information. Accommodation goes to the accommodation office. Anything else goes to general onboarding. The queue is a label for a person. It is not a dispatch into a live helpdesk system.

7. Memory, State and Interoperability

Two stores are easy to confuse, and the design keeps them apart. Session state lasts while the Streamlit session lasts. A reload clears the conversation, the agreement flag, and the active case id. The draft file on disk does not clear. That split is intentional. The chat is temporary. The staff draft is the record.

Persistent memory is Case History Recall, not a vector memory of the chat. If the current student message asks about a prior case, the registration number is taken only from the student's own messages, not from the assistant's text. get_case_history reads drafts.json by that exact identifier and injects a note for the rest of the turn. The group did not implement a

90-day deletion job. Drafts stay until a person removes the local file. The memory note records that limit. Drafts are gitignored so a registration number is not pushed to GitHub.

The interoperability artefact is an MCP-style interface specification for create_support_case, not a live Model Context Protocol server. The course reading did not cover MCP in the supplied editions, and the assignment allowed a clear interface design. The specification states the inputs, the validation errors, the queues, and the actions the tool will not perform. A later team could wrap the same handler as an MCP tool without changing the safety checks.

8. Evaluation, Observability and Guardrails

8.1 What was measured

Week 3 measured retrieval and grounded answers on 15 questions: answerable, partly answerable, and unanswerable. Week 7 measured the full agent on 36 scenarios, above the 30-scenario minimum. A scenario passed only when every applicable criterion passed: task completion, groundedness, tool choice, scope, latency under a 10-second target, and safety. The set mixes live Gemini calls and deterministic checks that use a fake client, so the safety paths do not depend on the model having a good day.

8.2 Results

Week 3: 11/15 (73%). Week 7: 29/36 (80.5%). Normal grounded questions and the adversarial injection cases passed. Limit checks passed, including the three-round stop. Incorrect-information cases passed at 80% because I02 missed the fee passage. Blocked-topic cases passed at 75% because two information questions were over-refused. Edge cases passed at 50% because a second-year student was treated as a freshman and a raw API error was shown in the chat. Latency sometimes exceeded 10 seconds: 13.6 seconds on a memory lookup and 12.5 seconds on case creation. Those turns still completed. The 10-second figure is a target, not a hard stop.

8.3 Guardrails

The guardrails are the Sense stops, the tool allow-list, schema validation, the agreement flag, the draft caps, and the rule that resolve, close, and reassign are not registered tools. Prompt injection is refused before tools run. A caller cannot supply a different session id and have it trusted. Fee-decision wording is rejected in the summary. The observability evidence and the guardrail evidence PDFs record these checks. Each v1.3 turn writes one log with the turn id, session id, latency, model and round count, prompt id, whether a case-history note was used, the student message, retrieval status and source titles, tool names and shortened results, the error if any, the stop condition, and a reply preview of at most 400 characters. Passage bodies and the full prompt are not stored.

8.4 Week 8 remediation and retest

Five failures were code defects and were fixed on tag v1.5.0. Information questions about the Code of Conduct or an admission letter are no longer blocked, while a request for admissions help or a grade change still stops at S5. A 503 or 429 is shown as "The assistant is temporarily busy. Please try again in a few minutes." The raw error remains in the log. If a draft was already saved, the reply still includes the case id and stops at S2. Second-year, third-year, fourth-year, fifth-year, and 2nd-year through 5th-year phrasing now stop at S4. Placeholder summaries, including "Unspecified onboarding issue" and summaries shorter than 8 characters, are validation errors. Fee-decision language is rejected in every category.

python run_deterministic_checks.py from src/ passed 16 of 16, including C07, C08, R04, and R05, which had encoded the expected behaviour and were failing before the fix. python week8_retest.py passed 5 of 5 on the code checks: the Code of Conduct question and the admission-letter question are not blocked, while an admissions decision, a grade change, and the second-year sentence still stop before the model. Those results are in docs/evaluation/week8_retest.json.

On 24 October 2026 the 19 live chats were run again from docs/evaluation/week8_runner_input.xlsx. The committed Week 7 results file has 35 scenarios, not the 36 named in the Week 7 write-up. The 29/36 figure is not rewritten. Eighteen chats finished. B07 now cites the Joining Instructions and lists the registration documents, stop S1. N03 now stops at S4 with no model call. I02 still returns no_match and stops at S3. B06 is no longer blocked in code, but the model refuses it and does not search, so the grounded-answer criterion still fails. M01, the memory question, stopped because Gemini returned 429 after the free-tier quota was used up. That turn is not scored as a product failure. Replies are in docs/evaluation/week8_live_results.csv. C05 saved one fees draft, CASE-2026-000001, in 11.4 seconds.

The checked-in traces are the inspection path when a marker cannot run Gemini. T1 is a grounded answer. T2 is a draft. T3 is a failure that recovers inside the limits. O1 is the admissions refusal at S5, with no model call. O2 is an instruction-override refusal at S6. O3 is a grounded webmail retrieval. During that run the model returned 503, so O3 records the retrieval result and says the model was unavailable. O4 keeps the live 503, including a latency of 8334 milliseconds, in the log rather than in the chat. Together they show a success, a hand-off, a blocked topic, an attack refused before tools, and a dependency failure that the application can name.

9. Software Engineering Quality and Deployment

The repository is https://github.com/SumayyaN/BSE41014_capstone_group_m_eve. History is on main through the Week 7 seal. The final release is annotated tag v1.5.0 on branch week8-release. Tag v1.3.0 is the Week 7 snapshot. Tag v1.4.0 is the README release before these defect fixes. A marker should check out v1.5.0. Run instructions are in the

README: Python 3.11 or newer, pip install -r requirements.txt, a GEMINI_API_KEY in .env copied from .env.example, the four RAG scripts, then python -m streamlit run src/app.py from the repository root.

Secrets stay out of git. .env is ignored. The sample file contains the variable name only. Drafts, embeddings, raw downloads, and logs are ignored. The checked-in traces are the Week 5 T1-T3 samples and the Week 7 O1-O4 samples. There is no continuous-integration workflow in the repository. The tests that travel with the release are the deterministic script and the Week 8 retest script. They do not need an API key. The live scenario runner does. The group did not deploy a public server. The reproducible run is the local Streamlit application from the tag. That matches the assignment option of a deployed or reproducibly runnable release.

Code review happened through pull requests on the shared repository, including the bounded-agent workflow, the memory work, and the observability logs. Week 8 fixes are small and sit next to the tests that name the old failures. A later maintainer can change a queue or a stop message without rewriting the model prompt, because those rules are tables and constants.

Reproducibility was treated as a deliverable, not as a hope. The README names tag v1.5.0, the Python version, the install command, the order of the four corpus scripts, and the Streamlit command. chunks.json is in git so the text of the corpus can be read before embeddings are rebuilt. The embedding file, raw downloads, drafts, and logs are generated on the machine that runs the demo. A second machine with the same key and the same tag should reach the same sidebar, the same two tools, and the same stop ids. It may not reach the same latency. The free-tier Gemini service has already returned 503 when demand was high, and a 429 when the daily embedding quota was exhausted. The embed script resumes after a 429. The chat path now explains a 503 without pasting it.

Maintainability also means a marker can see the limit without reading the chat code. STOP_CONDITIONS in orchestrate.py is the list S1 to S8. The failure catalogue names the six Week 7 misses and says which five were repaired. The README lists the two limits that remain: the 0.70 miss, and turns that can run past 10 seconds. A change to a queue name or a blocked phrase should be followed by the deterministic script. That script is the regression check for this release. It is not a substitute for a later live run, and the report does not pretend that it is.

10. Results, Limitations and Professional Reflection

What works is the path the product was built for. A freshman can ask how to activate webmail, see a cited answer when the passage clears 0.70, and, after an explicit click, leave one draft for staff. Injection attempts, grade changes, fee decisions, and non-freshman users are refused by code. A reader can inspect a turn log and see which stop fired and which tool

ran.

What does not work yet is complete recall of the corpus. Similarity at 0.70 misses some passages a person would judge relevant, especially fee policy and a few joining-instruction details. The safe response is a refusal, and that refusal can be wrong from the student's point of view. Latency can exceed 10 seconds when the model is slow or returns 503 under load. The student now sees a short retry message, but the answer is still delayed. There is no staff inbox, no authentication, and no deletion schedule for drafts. Those are product limits, not hidden features.

The professional point is that the model is not the system of record. A support answer that affects a fee, a place, or a case status has to be owned by a member of staff. The agent prepares a draft and shows its evidence. It does not close the case. Future work, if the university authorised it, would be a staff review screen, a retention period for drafts, and a retrieval evaluation before any change to the 0.70 cutoff. Until that evaluation exists, the cutoff stays.

Cost and latency belong in the same reflection. The project uses the Gemini free tier. There is no separate budget line and no paid deployment. A grounded turn is one embedding comparison locally plus one or two generate calls. Week 7 saw turns above the 10-second target, up to 13.6 seconds. That is acceptable for a demonstration and poor for a busy helpdesk. The response in this release is honesty in the log and a calm sentence in the chat, not a claim that the free tier is a production service. If the university later hosted this agent, the first work would be a staff queue with names on it, an authentication boundary, a retention rule for drafts, and a fresh retrieval score before anyone moved 0.70.

The demonstration should make the same distinction the report makes. A successful webmail answer shows the cited source. A turn log shows the retrieval count, the tool, and the stop id. The fee-refund question shows the remaining retrieval miss, or the second-year sentence shows the scope refusal that Week 8 closed. The agreement button shows the human gate. None of those moments require the agent to be trusted on tone. Each one can be checked against a file or a stop id.

References

Huyen, C. (2025). AI Engineering: Building Applications with Foundation Models. O'Reilly Media.

Lanham, M. (2025). AI Agents in Action. Manning Publications.

Osmani, A. (2025). Beyond Vibe Coding: From Coder to AI-Era Developer. O'Reilly Media.

Group M Evening. (2026). Repository evidence cited in the appendix: Agent Task Contract, prompt specification, tool catalogue, corpus source register, memory note, MCP-style interface specification, Week 3 rag_eval.md, Week 7 evaluation report, failure catalogue,

and observability samples. Makerere University, BSE4104. https://github.com/SumayyaN/BSE41014_capstone_group_m_eve

Appendix A. Evidence index

This appendix is outside the 8-12 page body. It points at files in tag v1.5.0.

- docs/requirements/Agent Task Contract M EVE.pdf
- docs/requirements/Group_M_User_Stories_and_Acceptance_Criteria.pdf
- docs/requirements/Group M - Prompt Specification v1.0.pdf
- docs/requirements/Tool_Catalogue_and_API_Schemas.pdf
- docs/requirements/Group M - Corpus Source Register.pdf
- docs/requirements/Memory_Design_and_Data_Handling_Note.pdf
- docs/requirements/MCP_Interface_Specification create_support_case.pdf
- docs/Architecture/Group_M_eve_State_Model.pdf
- docs/evaluation/rag_eval.md
- docs/evaluation/week7_evaluation.md
- docs/evaluation/failure_catalogue_week7.md
- docs/evaluation/week8_retest.json
- docs/evaluation/week8_live_results.csv
- docs/evaluation/week8_runner_input.xlsx
- docs/evaluation/Group_M_eve_Observability_Tracing_Evidence.pdf
- docs/evaluation/guardrail_Implementation_Evidence.pdf
- docs/evaluation/Group_M_eve_Final_Evidence_Pack.pdf
- evidence/traces/T1_grounded_answer.json, T2_case_draft.json, T3_failure_recovery.json
- evidence/traces/O1_blocked_topic.json through O4_model_unavailable.json
- src/run_deterministic_checks.py and src/week8_retest.py

Appendix B. Week 8 retest

Deterministic suite after the fixes: 16 PASS, 0 FAIL. The code retest lets B06 and B07 through and still blocks an admissions decision, a grade change, and a second-year sentence. The 24 October live run confirms B07 and N03. I02 still misses at 0.70. B06 is refused by the model. M01 did not finish because the Gemini quota returned 429.